



MANUAL TÉCNICO

Coodetrans Gestión

Sistema de escritorio para la administración de carpetas físicas, inventario de dotación,
movimientos internos y configuración organizacional

Versión del documento: 1.0

Versión del sistema documentado: 1.1.3

Cooperativa Coodetrans — Desarrollado por RickLabs

Autor original del software: Julian Ricardo Marcillo Tobar — C.C. 1080041352

GitHub: RickLabs

Control de versiones del documento

Versión	Fecha	Autor	Descripción del cambio
1.0	2026-09-05	Documentación técnica	Versión inicial del manual técnico, basada en el código fuente v1.1.3.

Autoría del software

Campo	Dato
Autor original del software	Julian Ricardo Marcillo Tobar
Cédula de ciudadanía	1080041352
Usuario / organización en GitHub	RickLabs
Cooperativa / cliente	Coodetrans

Tabla de contenido

Esta tabla se actualiza automáticamente al abrir el documento en Microsoft Word (clic derecho → Actualizar campos), o mediante F9.

Control de versiones del documento.....	2
Autoría del software	2
Tabla de contenido	3
1. Introducción.....	6
1.1 Propósito del documento	6
1.2 Alcance	6
1.3 Descripción general del sistema	6
1.4 Público objetivo	6
2. Tecnologías utilizadas	7
2.1 Requisitos de entorno de desarrollo.....	7
3. Arquitectura del sistema.....	8
3.1 Diagrama de capas.....	8
3.2 Responsabilidad de cada componente	8
3.2.1 Renderer (React)	8
3.2.2 Preload (puente seguro)	8
3.2.3 Main Process	8
3.2.4 Controladores	9
3.2.5 Repositorios	9
3.2.6 Modelos	9
3.2.7 Base de datos	9
3.3 Seguridad del proceso Electron	9
4. Estructura del proyecto	11
5. Modelo de datos.....	12
5.1 Ubicación del archivo de base de datos.....	12
5.2 Entidades y relaciones	12
5.2.1 Estructura organizacional	12
5.2.2 Catálogo central	12
5.2.3 Personal y seguridad	12
5.2.4 Inventario y movimientos	13
5.2.5 Auditoría	13
5.3 Migraciones no destructivas.....	13

5.4 Índices	13
6. Comunicación IPC (API interna)	15
6.1 Autenticación y aplicación	15
6.2 Dashboard y catálogos	15
6.3 Empleados, inventario y movimientos	15
6.4 Usuarios, roles, importación/exportación y base de datos	16
6.5 Eventos push (main → renderer)	16
7. Módulos funcionales (lógica de negocio)	18
7.1 Autenticación	18
7.2 Dashboard	18
7.3 Carpetas físicas (empleados)	18
7.4 Inventario	18
7.5 Movimientos y entregas	18
7.6 Configuración (usuarios, roles, áreas, cargos)	18
7.7 Importación y exportación	18
7.8 Base de datos (respaldo y restauración)	19
8. Flujos de proceso clave	20
8.1 Inicio de la aplicación	20
8.2 Registro de una entrega de dotación	20
8.3 Importación masiva de empleados	20
8.4 Copia de seguridad y restauración	20
8.5 Actualización automática	20
9. Seguridad	22
9.1 Contraseñas	22
9.2 Control de acceso y bloqueo	22
9.3 Aislamiento del proceso Electron	22
9.4 Firma de código	22
9.5 Aislamiento de datos	22
10. Puesta en marcha del entorno de desarrollo	23
10.1 Clonar el repositorio	23
10.2 Instalar dependencias	23
10.3 Recompilar el módulo nativo SQLite	23
10.4 Ejecutar en modo desarrollo	23
10.5 Usuario inicial	23
11. Compilación y empaquetado	24

11.1 Scripts disponibles	24
11.2 Salida de la compilación.....	24
11.3 Configuración de electron-builder (resumen)	24
11.4 Recursos extra empaquetados	24
12. Sistema de actualizaciones automáticas	26
12.1 Publicar una nueva versión.....	26
12.2 Comportamiento en el cliente	26
12.3 Preservación de datos.....	26
12.4 Solución de problemas.....	26
No detecta actualizaciones	26
Error al descargar	26
13. Mantenimiento y operación	27
13.1 Copias de seguridad.....	27
13.2 Restauración	27
13.3 Recomendaciones operativas	27
13.4 Scripts de utilidad (scripts/)	27
14. Anexos.....	28
14.1 Variables de entorno.....	28
14.2 Glosario	28
14.3 Documentación complementaria del repositorio.....	28

1. Introducción

1.1 Propósito del documento

Este manual técnico describe la arquitectura, el modelo de datos, los módulos funcionales y los procedimientos de instalación, compilación, distribución y mantenimiento del sistema Coodetrans Gestión. Está dirigido a desarrolladores, personal de soporte técnico y a cualquier persona responsable de mantener, extender o auditar la aplicación.

1.2 Alcance

El documento cubre la versión 1.1.3 del sistema, tal como se encuentra en el repositorio clonado en la ruta local del proyecto. Incluye la totalidad de las capas de la aplicación: interfaz de usuario (renderer), proceso principal de Electron, comunicación IPC, controladores de negocio, repositorios de datos y esquema de base de datos SQLite.

1.3 Descripción general del sistema

Coodetrans Gestión es una aplicación de escritorio multiplataforma (empaquetada actualmente para Windows x64) que permite a la Cooperativa Coodetrans administrar de forma local, sin dependencia de servicios externos ni conexión a internet permanente, los siguientes procesos:

- Control de carpetas físicas de empleados (hojas de vida, ubicación física, importación/exportación masiva).
- Inventario general de dotaciones, con control de stock por talla y variante.
- Historial de movimientos y entregas de dotación, con descuento automático de inventario.
- Configuración organizacional: usuarios, roles dinámicos, áreas, cargos.
- Copias de seguridad y restauración de la base de datos.
- Actualizaciones automáticas distribuidas mediante GitHub Releases.

1.4 Público objetivo

Desarrolladores que continúen el mantenimiento del proyecto, personal técnico de RickLabs y cualquier tercero autorizado que requiera comprender el funcionamiento interno del sistema.

2. Tecnologías utilizadas

La siguiente tabla resume el stack tecnológico principal, extraído directamente de package.json.

Tecnología	Versión	Uso en el proyecto
Electron	^33.2.1	Runtime de la aplicación de escritorio (proceso principal + ventana Chromium).
React	^18.3.1	Construcción de la interfaz de usuario (renderer).
React Router DOM	^6.28.0	Enrutamiento entre las páginas de la aplicación.
Tailwind CSS	^3.4.15	Sistema de estilos utilitario.
Vite	^6.0.3	Bundler y servidor de desarrollo del renderer.
better-sqlite3	^11.8.1	Driver nativo (síncrono) de acceso a la base de datos SQLite.
electron-updater	^6.6.2	Actualizaciones automáticas vía GitHub Releases.
electron-builder	^25.1.8	Empaquetado y generación de instaladores (NSIS, portable).
xlsx (SheetJS)	^0.18.5	Lectura y escritura de archivos Excel/CSV para importación y exportación.
recharts	^2.13.3	Gráficas del dashboard.
lucide-react	^0.460.0	Iconografía de la interfaz.

2.1 Requisitos de entorno de desarrollo

- Node.js 20 o superior.
- npm 10 o superior.
- Git.
- Windows 10 o superior para generar builds firmados/instaladores NSIS (build multiplataforma es posible en macOS/Linux para sus respectivos targets).

3. Arquitectura del sistema

La aplicación implementa una arquitectura en capas de tipo MVC adaptada al modelo multiproceso de Electron, separando estrictamente la interfaz de usuario, la lógica de negocio y la persistencia de datos.

3.1 Diagrama de capas



3.2 Responsabilidad de cada componente

3.2.1 Renderer (React)

Ubicado en `src/renderer`. Contiene las páginas (`pages/`), componentes reutilizables (`components/`), layout y barra lateral (`layouts/`), hooks personalizados (`hooks/`) y utilidades (`utils/`). Es responsable de renderizar las interfaces, realizar validaciones básicas de formulario, gestionar la navegación (React Router) y consumir exclusivamente la API expuesta por el preload a través de `window.api`. El renderer nunca importa módulos de Node.js ni accede directamente a la base de datos.

3.2.2 Preload (puente seguro)

`src/main/preload.js` utiliza `contextBridge.exposeInMainWorld` para publicar un objeto `window.api` con métodos agrupados por dominio (`auth`, `dashboard`, `catalogos`, `empleados`, `inventario`, `movimientos`, `usuarios`, `roles`, `importExport`, `bd`, `app`, `update`). Cada método invoca un canal IPC específico mediante `ipcRenderer.invoke`. Esta es la única superficie de comunicación entre el mundo del renderer (`sandboxed`, sin Node) y el proceso principal.

3.2.3 Main Process

`src/main/main.js` crea la ventana principal (`BrowserWindow`), inicializa la base de datos, registra los manejadores IPC y gestiona el ciclo de vida de la aplicación (apertura, cierre, activación en macOS).

además del sistema de actualizaciones automáticas (electron-updater). `src/main/ipc-handlers.js` centraliza el registro de todos los canales `ipcMain.handle(...)` y delega cada uno en el controlador correspondiente, devolviendo siempre un objeto con la forma `{ ok, data?, error? }`.

3.2.4 Controladores

`src/controllers/controllers.js` concentra la lógica de negocio y las validaciones: autenticación, dashboard, catálogos (áreas/cargos/ubicaciones), empleados, inventario, movimientos/entregas, usuarios, roles, importación/exportación y administración de la base de datos. Orquestan uno o varios repositorios y aplican las reglas de negocio (por ejemplo, descontar stock al registrar una entrega, o impedir eliminar un área con cargos asociados).

3.2.5 Repositorios

`src/repositories/repositories.js` encapsula el acceso a SQLite mediante sentencias preparadas (better-sqlite3), exponiendo funciones puras de consulta e inserción/actualización por entidad (usuarioRepo, rolRepo, areaRepo, cargoRepo, ubicacionRepo, empleadoRepo, articuloRepo, entregaRepo, actividadRepo). No contienen lógica de negocio, solo acceso a datos.

3.2.6 Modelos

`src/models/models.js` define la forma de referencia de cada entidad del dominio y aporta funciones auxiliares de transformación/derivación, como el cálculo del estado de stock (normal / bajo / crítico) o la construcción de la etiqueta legible de una variante de talla.

3.2.7 Base de datos

`src/database/database.js` gestiona la conexión única (singleton) a SQLite mediante better-sqlite3, aplica el esquema (schema.sql), ejecuta migraciones no destructivas y expone utilidades para copia de seguridad y restauración segura de archivos.

3.3 Seguridad del proceso Electron

La ventana principal se crea con la siguiente configuración de seguridad (`src/main/main.js`):

Opción	Valor	Efecto
contextIsolation	true	Aísla el contexto de JavaScript del renderer del contexto de preload/Node.
nodeIntegration	false	El renderer no tiene acceso directo a los módulos de Node.js.
sandbox	true	Ejecuta el proceso de renderizado en un sandbox del sistema operativo.
enableRemoteModule	false	Deshabilita el módulo remoto (obsoleto e inseguro).
webSecurity	true	Mantiene las políticas de seguridad web estándar (CORS, etc.).

Opción	Valor	Efecto
Nota de seguridad Toda comunicación entre el renderer y el sistema de archivos, la base de datos o los diálogos nativos pasa obligatoriamente por un canal IPC explícito registrado en ipc-handlers.js. No existe ninguna vía alterna de acceso desde la interfaz.		

4. Estructura del proyecto

Árbol de directorios principal del repositorio (D:\DESARROLLO_WEB\coodetrans):

coodetrans/	
├─ build/	Recursos de empaquetado (íconos, licencia)
├─ docs/	Documentación complementaria (.md) y plantilla de
importación	
├─ scripts/	Scripts de utilidad (rebuild nativo, firma, íconos,
pruebas)	
├─ src/	
├─ config/	Configuración general (usuario inicial, roles iniciales)
├─ controllers/	Lógica de negocio y validaciones
├─ database/	Conexión SQLite, esquema, migraciones, semilla
└─ main/	Proceso principal de Electron (main, preload, ipc-
handlers)	
├─ models/	Definición de entidades y helpers de dominio
├─ renderer/	Aplicación React (UI)
├─ assets/	Imágenes y logotipos
├─ components/	Componentes reutilizables (Tabla, Modal, Botón, etc.)
├─ hooks/	useAuth, useUpdater, useAppVersion, useDebounceValue
├─ layouts/	Layout general y barra lateral (Sidebar)
└─ pages/	Páginas: Login, Dashboard, Carpetas, Inventario,
Movimientos, Configuración	
├─ utils/	Utilidades del renderer (wrapper de window.api)
├─ repositories/	Acceso a datos (SQL preparado)
└─ services/	Servicios de dominio transversal (importExportService)
├─ package.json	Scripts npm y configuración de electron-builder
├─ vite.config.mjs	Configuración del bundler del renderer
├─ tailwind.config.cjs	Configuración de estilos
└─ CHANGELOG.md, README.md, GUIA_EJECUCION_Y_DESPLIEGUE.md	

5. Modelo de datos

Motor: SQLite 3, accedido mediante el driver nativo better-sqlite3. El esquema completo se define en `src/database/schema.sql` y se ejecuta automáticamente (CREATE TABLE IF NOT EXISTS) en cada arranque de la aplicación, junto con un conjunto de migraciones no destructivas (ver sección 5.3).

5.1 Ubicación del archivo de base de datos

La base de datos se almacena como archivo único `coodetrans.db` dentro de la carpeta `userData` que gestiona Electron:

Windows: `%APPDATA%\Coodetrans Gestión\coodetrans.db`

Configuración de PRAGMAs aplicada al abrir la conexión (`src/database/database.js`): `journal_mode = WAL`, `foreign_keys = ON`, `synchronous = NORMAL`, `cache_size = -64000`, `temp_store = MEMORY`. Estos ajustes priorizan la integridad referencial y el rendimiento en lecturas/escrituras concurrentes propias del modo WAL.

5.2 Entidades y relaciones

5.2.1 Estructura organizacional

Tabla	Descripción	Relaciones
area	Áreas de la cooperativa (ej. Operaciones, Administración).	Referenciada por cargo, empleado y artículo.
cargo	Cargos dentro de un área.	FK → area (RESTRICT en borrado).

5.2.2 Catálogo central

Tabla	Descripción
ubicacion_fisica	Catálogo histórico de ubicaciones (compatibilidad; el dato vigente vive como texto libre en <code>empleado.ubicacion_fisica</code>).
talla	Variantes de talla: camisa, pantalón, calzado, o una categoría general con nombre descriptivo.

5.2.3 Personal y seguridad

Tabla	Descripción	Notas clave
rol	Roles dinámicos administrables desde Configuración → Roles.	nombre UNIQUE, estado Activo/Inactivo.
empleado	Hoja de vida física del empleado.	estado restringido a Activo/Retirado (CHECK). Ubicación física en texto libre. <code>fk_id_area</code> independiente del cargo para soportar importaciones parciales.

Tabla	Descripción	Notas clave
usuario	Cuentas de acceso al sistema.	password almacenada como hash; fk_id_empleado puede ser NULL (usuario sin empleado asociado, p. ej. "Programador").

5.2.4 Inventario y movimientos

Tabla	Descripción	Notas clave
articulo	Ítem de dotación (ej. camisa, pantalón).	stock_minimo (umbral de alerta) y vencimiento (0/1).
articulo_talla_stock	Stock exacto por combinación artículo + talla.	UNIQUE(fk_id_articulo, fk_id_talla). Es la única fuente de verdad del stock.
entrega_dotacion	Encabezado de una entrega a un empleado.	periodo restringido a Abril/Agosto/Diciembre (CHECK).
detalle_entrega	Líneas de una entrega (artículo/talla/cantidad).	cantidad > 0 (CHECK). Al confirmarse, descuenta articulo_talla_stock.

5.2.5 Auditoría

La tabla actividad_reciente no existe en el esquema MySQL original: se agregó específicamente para alimentar el dashboard y el historial global de movimientos de la aplicación de escritorio. Registra acción (creación, actualización, entrega, eliminación, sistema), entidad afectada, empleado y usuario responsables, y fecha.

5.3 Migraciones no destructivas

La función migrar() en database.js se ejecuta en cada arranque y aplica, solo si es necesario, los siguientes cambios sobre bases de datos creadas con versiones anteriores del esquema:

- Agrega la columna estado (Activo/Inactivo) a area, cargo y usuario si no existe.
- Agrega ubicacion_fisica (texto libre) y fk_id_area a empleado, migrando datos desde las tablas relacionadas cuando es posible.
- Reconstruye la tabla usuario para permitir fk_id_empleado NULL (usuarios sin empleado asociado).
- Reconstruye la tabla empleado para permitir fk_id_ubicacion NULL (ubicación pasa a texto libre).
- Crea índices adicionales de rendimiento sobre empleado.

Principio de diseño

Todas las migraciones están diseñadas para nunca eliminar datos existentes del usuario; solo agregan columnas o relajan restricciones, preservando la información entre versiones de la aplicación.

5.4 Índices

Se definen índices sobre las columnas más consultadas para acelerar filtros y búsquedas: nombre y cédula de empleado, estado de empleado/área/cargo/usuario, claves foráneas de área en

cargo/empleado/artículo, nombre de artículo, y fecha/periodo/empleado en entregas y detalle de entrega.

6. Comunicación IPC (API interna)

Toda interacción entre la interfaz (renderer) y la lógica de negocio (main) ocurre a través de canales IPC invocados con `ipcRenderer.invoke` y atendidos con `ipcMain.handle`. La siguiente tabla resume los canales agrupados por dominio, tal como están definidos en `src/main/ipc-handlers.js` y expuestos en `src/main/preload.js`.

6.1 Autenticación y aplicación

Canal	window.api	Descripción
auth:login	auth.login(credenciales)	Valida usuario/contraseña y devuelve el usuario (sin password) si son correctos.
app:get-version	app.version()	Devuelve la versión actual de la aplicación empaquetada.
update:install	update.instalar()	Aplica la actualización descargada y reinicia la aplicación.

6.2 Dashboard y catálogos

Canal	window.api	Descripción
dashboard:resumen	dashboard.obtenerResumen()	Indicadores generales para la pantalla principal.
catalogo:areas / areasActivas	catalogos.areas() / areasActivas()	Listado de áreas (todas / solo activas).
catalogo:cargos / cargosActivos	catalogos.cargos() / cargosActivos()	Listado de cargos.
catalogo:ubicaciones	catalogos.ubicaciones()	Catálogo histórico de ubicaciones físicas.
area:crear / area:actualizar / area:estado	catalogos.crearArea / actualizarArea / cambiarEstadoArea	CRUD de áreas.
cargo:crear / cargo:actualizar / cargo:estado	catalogos.crearCargo / actualizarCargo / cambiarEstadoCargo	CRUD de cargos.

6.3 Empleados, inventario y movimientos

Canal	window.api	Descripción
empleado:listar / obtener / crear / actualizar / historial	empleados.*	CRUD de empleados y consulta de su historial de entregas.
inventario:listar / resumen / variantes	inventario.listar / resumen / variantes	Consulta de inventario, resumen por área y listado de variantes de talla.

Canal	window.api	Descripción
inventario:crearArticulo / actualizarArticulo / eliminarArticulo	inventario.crearArticulo / actualizarArticulo / eliminarArticulo	CRUD de artículos de dotación.
inventario:crearVariante / ajustarStock / eliminarVariante	inventario.crearVariante / ajustarStock / eliminarVariante	Administración de stock por talla.
movimiento:listar	movimientos.listar(limite)	Últimos movimientos registrados en actividad_reciente.
entrega:listar / entrega:detalle / entrega:crear	movimientos.listarEntregas / detalleEntrega / crearEntrega	Consulta y registro de entregas de dotación.

6.4 Usuarios, roles, importación/exportación y base de datos

Canal	window.api	Descripción
usuario:listar / crear / actualizar / estado	usuarios.*	CRUD de usuarios del sistema.
rol:listar / listarActivos / crear / actualizar / estado	roles.*	CRUD de roles dinámicos.
import:seleccionar / previsualizar / confirmar	importExport.seleccionarArchivo / previsualizar / confirmar	Flujo de importación de empleados (Excel/CSV) con previsualización y confirmación.
export:empleados	importExport.exportar(filtro, formato, area, cargo, columnas)	Exporta empleados a Excel o CSV mediante diálogo nativo de guardado.
template:download	importExport.descargarPlantilla()	Copia la plantilla oficial de importación al destino elegido por el usuario.
db:info / db:backup / db:restore	bd.info / bd.backup / bd.restore	Información, copia de seguridad y restauración de la base de datos.

6.5 Eventos push (main → renderer)

Además de los canales invoke/handle, el proceso principal envía eventos asíncronos al renderer mediante webContents.send, escuchados con ipcRenderer.on en el preload:

Evento	Origen	Uso
db:restored	reiniciarTrasRestauracionBD()	Notifica al renderer que la base de datos fue restaurada y se recargará la ventana.
update:available	autoUpdater "update-available"	Notifica que hay una nueva versión disponible (versión, fecha, notas).
update:download-progress	autoUpdater "download-progress"	Progreso de descarga de la actualización (% , velocidad, bytes).

Evento	Origen	Uso
update:downloaded	autoUpdater "update-downloaded"	La actualización terminó de descargarse y está lista para instalar.
update:error	autoUpdater "error"	Error durante el proceso de comprobación/descarga de actualización.

Convención de retorno: todos los manejadores IPC invocados con `invoke()` devuelven un objeto homogéneo `{ ok: boolean, data?: any, error?: string }`, lo que permite al renderer manejar éxito y error de forma consistente en toda la aplicación.

7. Módulos funcionales (lógica de negocio)

7.1 Autenticación

`authController.login` valida usuario y contraseña contra la tabla usuario. La contraseña se compara mediante hash (ver sección 9.1). Si el usuario no existe o la contraseña no coincide, incrementa el contador de intentos fallidos y devuelve un error genérico ("Usuario o contraseña incorrectos") sin revelar cuál de los dos datos falló. Si el usuario está marcado como Inactivo, se rechaza el acceso aunque la contraseña sea correcta. En un inicio de sesión exitoso se actualiza `ultimo_acceso` y se registra la actividad en `actividad_reciente`. La contraseña (hash) nunca se devuelve al renderer.

7.2 Dashboard

`dashboardController.obtenerResumen` agrega indicadores generales (totales de empleados activos/retirados, estado del inventario por área, últimos movimientos) para la pantalla principal (`DashboardPage.jsx`), apoyándose en los helpers de `models.js` (`estadoStock`, `resumenStockPorArea`).

7.3 Carpetas físicas (empleados)

`empleadoController` gestiona el ciclo de vida de la hoja de vida física de cada empleado: listar, obtener el detalle, crear, actualizar y consultar el historial de entregas asociado. La página `CarpetasPage.jsx` es la más extensa de la interfaz (integra tabla, filtros, formularios, importación y exportación).

7.4 Inventario

`inventarioController` administra artículos de dotación y sus variantes de talla. El stock real vive en `articulo_talla_stock`; los ajustes de stock (`ajustarStock`) se registran también en `actividad_reciente` para trazabilidad. `estadoStock()` clasifica cada variante como normal, bajo o crítico según el porcentaje respecto al stock mínimo configurado.

7.5 Movimientos y entregas

`movimientoController.crearEntrega` registra una entrega de dotación a un empleado (`entrega_dotacion + detalle_entrega`) y descuenta automáticamente el stock correspondiente en `articulo_talla_stock` dentro de la misma transacción, garantizando que el inventario y el historial nunca queden desincronizados.

7.6 Configuración (usuarios, roles, áreas, cargos)

`usuarioController` y `rolController` exponen el CRUD de cuentas y roles dinámicos. `catalogoController` hace lo propio para áreas y cargos. Todas las operaciones de creación/actualización reciben `idUsuario` para dejar trazabilidad de quién realizó el cambio en `actividad_reciente`.

7.7 Importación y exportación

importExportController delega en `src/services/importExportService.js`, que utiliza la librería `xlsx` (SheetJS) para leer archivos `.xlsx/.xls/.csv`. El flujo es: seleccionar archivo → previsualizar y validar filas (documento, nombre, área, cargo, estado) → confirmar importación. Si la cédula ya existe, el registro se actualiza; si no, se crea un nuevo empleado. La exportación permite filtrar por todos/activos/retirados y elegir columnas y formato (Excel o CSV). La plantilla oficial se distribuye desde `docs/plantilla-importacion-datos.xlsx` (empaquetada como recurso extra en el instalador).

7.8 Base de datos (respaldo y restauración)

`dbController.backup()` cierra de forma segura la conexión (checkpoint WAL + eliminación de archivos `-wal/-shm`) antes de copiar el archivo `.db` al destino elegido por el usuario. `dbController.restore()` reemplaza el archivo activo por el seleccionado y dispara `reiniciarTrasRestauracionBD()`, que reabre la conexión y envía el evento `db:restored` a todas las ventanas para forzar una recarga suave (sin cerrar la sesión del usuario).

8. Flujos de proceso clave

8.1 Inicio de la aplicación

- 1. Electron dispara `app.whenReady()`.
- 2. Se inicializa la base de datos (`initDatabase`): se crea/verifica el esquema, se aplican migraciones y se ejecuta el seed inicial si las tablas están vacías.
- 3. Se registran todos los manejadores IPC (`registrarHandlersIPC`).
- 4. Se crea la ventana principal (`createWindow`) y se carga el renderer (Vite en desarrollo, `dist-renderer/index.html` en producción).
- 5. En producción, una vez la ventana está lista, se dispara `autoUpdater.checkForUpdates()`.

8.2 Registro de una entrega de dotación

- 1. El usuario selecciona empleado, periodo y artículos/tallas/cantidades en `MovimientosPage.jsx`.
- 2. El renderer invoca `movimientos.crearEntrega(datos, idUsuario) → canal entrega:crear`.
- 3. `movimientoController.crearEntrega` valida los datos, inserta `entrega_dotacion` y sus `detalle_entrega`, y descuenta el stock en `articulo_talla_stock` dentro de una transacción SQLite.
- 4. Se registra la actividad en `actividad_reciente`.
- 5. La UI refresca inventario y movimientos.

8.3 Importación masiva de empleados

- 1. El usuario elige "Importar" en `CarpetasPage.jsx` → se abre el diálogo nativo (`import:seleccionar`).
- 2. El archivo se procesa con `importExportService` (lectura `xlsx/csv`) y se devuelve una previsualización con filas válidas e inválidas.
- 3. El usuario revisa los errores señalados y confirma → `import:confirmar`.
- 4. Por cada fila válida: si la cédula existe se actualiza el empleado, si no existe se crea uno nuevo.

8.4 Copia de seguridad y restauración

- 1. Copia de seguridad: `db:backup` → `checkpoint WAL`, cierre de conexión, copia del archivo `.db` al destino elegido, reapertura de la conexión.
- 2. Restauración: `db:restore` → confirmación explícita del usuario (diálogo de advertencia) → reemplazo del archivo `.db` → `reiniciarTrasRestauracionBD()` → evento `db:restored` → recarga de la ventana sin perder la sesión activa.

8.5 Actualización automática

- 1. Al arrancar en producción, `autoUpdater.checkForUpdates()` consulta la última Release publicada en GitHub (RickJRMT/coodetrans-gestion).

- 2. Si hay una versión nueva, se descarga en segundo plano y se notifica el progreso a la ventana (update:download-progress).
- 3. Al finalizar la descarga se emite update:downloaded; el usuario decide cuándo instalar (update:install → autoUpdater.quitAndInstall()).
- 4. Los datos del usuario (empleados, inventario, configuración, base SQLite) residen en AppData y nunca se ven afectados por el proceso de actualización.

9. Seguridad

9.1 Contraseñas

Las contraseñas se almacenan con hash SHA-256 (función `hashPassword` en `src/database/seed.js`), con el prefijo `"sha256$"` seguido del hash hexadecimal.

Deuda técnica documentada por el propio equipo de desarrollo

El código señala explícitamente en un comentario que SHA-256 sin sal es un "placeholder" y que en una fase posterior debería reemplazarse por BCrypt (o un algoritmo equivalente con sal e iteraciones, como Argon2 o scrypt), sin afectar el resto del sistema ya que el hash se calcula en un único punto (`hashPassword`). Se recomienda priorizar esta migración antes de un despliegue con datos sensibles adicionales.

9.2 Control de acceso y bloqueo

Cada intento de inicio de sesión fallido incrementa `intentos_fallidos` en la tabla `usuario` (`usuarioRepo.incrementarFallidos`). Los usuarios con estado distinto de "Activo" no pueden iniciar sesión aunque la contraseña sea correcta. La aplicación no implementa actualmente un bloqueo automático tras N intentos fallidos; el campo existe como base para dicha regla si se desea añadir en el futuro.

9.3 Aislamiento del proceso Electron

Ver sección 3.3. En resumen: `contextIsolation`, `sandbox` y deshabilitación de `nodeIntegration` garantizan que el código de la interfaz (React) no tenga acceso directo a Node.js, al sistema de archivos ni a la base de datos; toda operación sensible pasa por un canal IPC explícito y auditable.

9.4 Firma de código

Desde la versión 1.1.3 el proceso de build incluye una verificación previa de firma digital (`scripts/check-signing.js`, ejecutado como `check:signing` antes de cada build) y soporta firma de los ejecutables generados. Las credenciales y certificados se gestionan mediante variables de entorno en un archivo `.env` (no versionado; se distribuye `.env.example` como plantilla). El objetivo declarado en el CHANGELOG es migrar en el futuro a certificados emitidos por una Autoridad Certificadora oficial.

9.5 Aislamiento de datos

La base de datos SQLite es local a cada instalación (carpeta `userData`) y no se sincroniza ni transmite a ningún servidor externo. La única comunicación de red de la aplicación es la verificación/descarga de actualizaciones contra GitHub Releases.

10. Puesta en marcha del entorno de desarrollo

10.1 Clonar el repositorio

```
git clone <url-del-repositorio>
cd coodetrans-gestion
```

10.2 Instalar dependencias

```
npm install
```

El script postinstall (scripts/ensure-native-deps.js) se ejecuta automáticamente tras la instalación para verificar las dependencias nativas.

10.3 Recompilar el módulo nativo SQLite

better-sqlite3 es un módulo nativo que debe compilarse contra la versión exacta de Electron utilizada por el proyecto:

```
npm run rebuild
```

10.4 Ejecutar en modo desarrollo

El siguiente comando levanta en paralelo el servidor de desarrollo de Vite (renderer, puerto 5173) y el proceso de Electron, que espera a que el puerto esté disponible antes de arrancar:

```
npm run dev
```

Scripts equivalentes por separado, útiles para depuración:

```
npm run dev:renderer    # Solo Vite
npm run dev:electron    # Solo Electron (requiere Vite ya corriendo)
```

10.5 Usuario inicial

La primera vez que se ejecuta la aplicación (base de datos vacía), el sistema crea automáticamente un único usuario:

Campo	Valor
Usuario	Programador
Contraseña	Coodetrans2026 (o el valor de la variable de entorno COODETRANS_DEV_PASSWORD)
Rol	Desarrollador
Recomendación Cambiar la contraseña del usuario "Programador" inmediatamente después del primer ingreso, o definir la variable de entorno COODETRANS_DEV_PASSWORD antes del primer arranque en cada instalación productiva.	

11. Compilación y empaquetado

11.1 Scripts disponibles

Comando	Resultado
npm run build:renderer	Compila únicamente el frontend (Vite) a dist-renderer/.
npm run build	Verifica firma, prepara íconos, compila el renderer y genera el instalador Windows x64 (NSIS + portable, según configuración de "win.target").
npm run build:portable	Genera únicamente la versión portable (.exe sin instalador).
npm run build:installer	Genera únicamente el instalador NSIS.
npm run build:signed / build:signed:installer	Igual que los anteriores, forzando el flujo de verificación de firma.
npm run build:all	Build multiplataforma (Windows, macOS, Linux) con electron-builder - mwl.

Todos los scripts de build ejecutan primero check:signing (validación de firma) y prepare:icons (generación de íconos .ico/.png a partir de la fuente en build/) antes de invocar a electron-builder.

11.2 Salida de la compilación

Los artefactos generados (instalador .exe NSIS, ejecutable portable, archivos .blockmap y latest.yml para el actualizador) se almacenan en:

```
dist/
```

11.3 Configuración de electron-builder (resumen)

Clave	Valor	Notas
appId	com.ricklabs.coodetrans.gestion	Identificador único de la aplicación.
asar	true	El código fuente se empaqueta en un archivo .asar.
asarUnpack	better-sqlite3/**	El binario nativo se extrae del asar para poder cargarse en tiempo de ejecución.
win.target	portable + nsis (x64)	Genera ambos formatos de distribución para Windows.
nsis.deleteAppDataOnUninstall	false	Al desinstalar, los datos del usuario (AppData) se conservan.
publish.provider	github (RickJRM/ct-coodetrans-gestion)	Repositorio usado por electron-updater para buscar nuevas versiones.

11.4 Recursos extra empaquetados

Además del código de la aplicación, el build incluye como "extraResources" (accesibles en tiempo de ejecución vía `process.resourcesPath`): `src/database/schema.sql`, los íconos `build/icon.ico` e `icon.png`, y la plantilla `docs/plantilla-importacion-datos.xlsx` utilizada por la función de descarga de plantilla de importación.

12. Sistema de actualizaciones automáticas

Coodetrans Gestión utiliza electron-updater sobre GitHub Releases del repositorio RickJRMT/coodetrans-gestion.

12.1 Publicar una nueva versión

- 1. Incrementar el campo "version" en package.json (ej. 1.1.3 → 1.1.4).
- 2. Generar el build de producción: npm run build.
- 3. Crear una nueva Release en GitHub con el tag correspondiente (ej. v1.1.4).
- 4. Adjuntar a la Release los artefactos generados en dist/: el/los instalador(es) .exe, el archivo .exe.blockmap y, de forma imprescindible, latest.yml (metadatos que consulta el cliente para saber si hay una versión nueva).
- 5. Publicar la Release.

12.2 Comportamiento en el cliente

Al iniciar en producción, la aplicación consulta la última Release, y si detecta una versión superior a la instalada la descarga automáticamente en segundo plano, notificando el progreso a través del componente UpdateNotification.jsx. Una vez descargada, el usuario decide cuándo reiniciar para aplicar la instalación.

12.3 Preservación de datos

El proceso de actualización nunca elimina ni modifica el contenido de la carpeta userData: empleados, inventario, configuración, usuarios y la base de datos SQLite completa permanecen intactos entre versiones.

12.4 Solución de problemas

No detecta actualizaciones

- Verificar que la Release esté publicada (no en borrador) y que incluya latest.yml.
- Confirmar que publish.owner/repo en package.json coincide con el repositorio real.

Error al descargar

- Verificar la conexión a internet del equipo.
- Revisar los permisos/visibilidad del repositorio de GitHub (debe ser accesible para electron-updater).

13. Mantenimiento y operación

13.1 Copias de seguridad

Disponibles desde Configuración → Base de Datos dentro de la aplicación. El proceso realiza un checkpoint WAL y cierra la conexión antes de copiar el archivo .db, evitando copias corruptas o incompletas.

13.2 Restauración

Al restaurar, se solicita confirmación explícita al usuario (la operación reemplaza todos los datos actuales). Tras la restauración, la conexión se reabre y todas las ventanas reciben el evento db:restored, que dispara una recarga suave de la interfaz sin cerrar la sesión activa ni requerir reiniciar el proceso completo de Electron.

13.3 Recomendaciones operativas

- Realizar copias de seguridad periódicas, especialmente antes de importaciones masivas o actualizaciones de versión mayor.
- No modificar el archivo coodetrans.db manualmente con herramientas externas; utilizar únicamente las funciones de la aplicación.
- Mantener la variable de entorno COODETRANS_DEV_PASSWORD (o la contraseña del usuario Programador) bajo control del área de sistemas.
- Revisar el CHANGELOG.md antes de cada actualización para conocer cambios relevantes de comportamiento.

13.4 Scripts de utilidad (scripts/)

Script	Propósito
ensure-native-deps.js	Verifica dependencias nativas tras npm install (postinstall).
check-signing.js	Valida la configuración de firma digital antes de compilar.
prepare-icons.js	Genera los íconos .ico/.png usados en el build a partir de la fuente gráfica.
test-import-export.js	Prueba end-to-end del flujo de importación/exportación de empleados.
temp-export-test.js	Prueba puntual de exportación.

14. Anexos

14.1 Variables de entorno

Variable	Uso
COODETRANS_DEV_PASSWORD	Sobrescribe la contraseña por defecto del usuario inicial "Programador".
NODE_ENV	development activa el modo de desarrollo (carga desde Vite, DevTools).
(.env, no versionado)	Credenciales y certificados usados por el proceso de firma de código (ver .env.example para la plantilla de variables requeridas).

14.2 Glosario

Término	Definición
IPC	Inter-Process Communication: mecanismo de Electron para comunicar el proceso principal (Node.js) con el proceso de renderizado (Chromium/React).
contextBridge	API de Electron para exponer de forma segura y controlada funciones del proceso principal al renderer.
WAL	Write-Ahead Logging: modo de journaling de SQLite que mejora la concurrencia de lectura/escritura.
ASAR	Formato de archivo empaquetado usado por Electron para distribuir el código fuente de la aplicación.
NSIS	Nullsoft Scriptable Install System: sistema usado para generar el instalador de Windows.

14.3 Documentación complementaria del repositorio

- README.md — visión general y guía rápida.
- docs/arquitectura.md, docs/base-de-datos.md, docs/instalacion.md, docs/actualizaciones.md, docs/importacion-exportacion.md.
- CHANGELOG.md — historial detallado de versiones.
- CONTRIBUTING.md — normas para contribuir al proyecto.
- GUIA_EJECUCION_Y_DESPLIEGUE.md — referencia rápida de comandos.